

```
# Step 1 – Import libraries
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

```
# Step 2 – Load dataset
```

```
df = pd.read_csv("pasteurized_milk_shelf_life_dataset.csv")
```

```
df.head(10)
```

|   | Storage_Temperature_C | Storage_Time_Days | pH    |
|---|-----------------------|-------------------|-------|
| 0 | 2.73                  | 5.40              | 6.519 |
| 1 | 3.44                  | 4.24              | 6.555 |
| 2 | 2.73                  | 9.29              | 6.391 |
| 3 | 6.59                  | 5.52              | 6.438 |
| 4 | 6.47                  | 3.31              | 6.634 |
| 5 | 5.63                  | 6.33              | 6.408 |
| 6 | 5.34                  | 1.09              | 6.608 |
| 7 | 4.41                  | 0.52              | 6.701 |
| 8 | 8.24                  | 12.10             | 6.131 |
| 9 | 2.78                  | 8.22              | 6.479 |

|   | Microbial_Load_log_CFU_mL | Sensory_Score | Shelf_Life_Status |
|---|---------------------------|---------------|-------------------|
| 0 | 2.65                      | 8.77          | Acceptable        |
| 1 | 3.04                      | 8.43          | Acceptable        |
| 2 | 3.66                      | 7.66          | Acceptable        |
| 3 | 3.02                      | 8.38          | Acceptable        |
| 4 | 2.84                      | 8.15          | Acceptable        |
| 5 | 3.36                      | 7.08          | Acceptable        |
| 6 | 2.42                      | 9.37          | Acceptable        |

|   |      |      |            |
|---|------|------|------------|
| 7 | 2.16 | 9.68 | Acceptable |
| 8 | 4.76 | 5.97 | Spoiled    |
| 9 | 3.34 | 7.70 | Acceptable |

*# Step 3 – Understand the dataset*

```
print(df.shape)
print(df.info())
print(df.describe())
```

```
(1500, 7)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 7 columns):
```

| # | Column                    | Non-Null Count | Dtype   |
|---|---------------------------|----------------|---------|
| 0 | Storage_Temperature_C     | 1500 non-null  | float64 |
| 1 | Storage_Time_Days         | 1500 non-null  | float64 |
| 2 | pH                        | 1500 non-null  | float64 |
| 3 | Titratable_Acidity_pct    | 1500 non-null  | float64 |
| 4 | Microbial_Load_log_CFU_mL | 1500 non-null  | float64 |
| 5 | Sensory_Score             | 1500 non-null  | float64 |
| 6 | Shelf_Life_Status         | 1500 non-null  | object  |

dtypes: float64(6), object(1)

memory usage: 82.2+ KB

None

|       | Storage_Temperature_C | Storage_Time_Days | pH          | \ |
|-------|-----------------------|-------------------|-------------|---|
| count | 1500.000000           | 1500.000000       | 1500.000000 |   |
| mean  | 5.995080              | 7.506393          | 6.401122    |   |
| std   | 2.352092              | 4.340732          | 0.163285    |   |
| min   | 2.040000              | 0.000000          | 6.002000    |   |
| 25%   | 3.900000              | 3.857500          | 6.271000    |   |
| 50%   | 6.055000              | 7.575000          | 6.402500    |   |
| 75%   | 8.032500              | 11.315000         | 6.535250    |   |
| max   | 10.000000             | 14.980000         | 6.775000    |   |

|               | Titratable_Acidity_pct | Microbial_Load_log_CFU_mL |
|---------------|------------------------|---------------------------|
| Sensory_Score |                        |                           |
| count         | 1500.000000            | 1500.000000               |
| 1500.000000   |                        |                           |
| mean          | 0.201523               | 3.580987                  |
| 7.316360      |                        |                           |
| std           | 0.039035               | 0.861092                  |
| 1.295568      |                        |                           |
| min           | 0.119000               | 1.660000                  |
| 4.050000      |                        |                           |
| 25%           | 0.169000               | 2.900000                  |
| 6.360000      |                        |                           |
| 50%           | 0.203000               | 3.600000                  |
| 7.295000      |                        |                           |

|           |          |          |
|-----------|----------|----------|
| 75%       | 0.232000 | 4.250000 |
| 8.310000  |          |          |
| max       | 0.286000 | 5.600000 |
| 10.000000 |          |          |

*# Step 4 – Check missing values and duplicates*

```
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
```

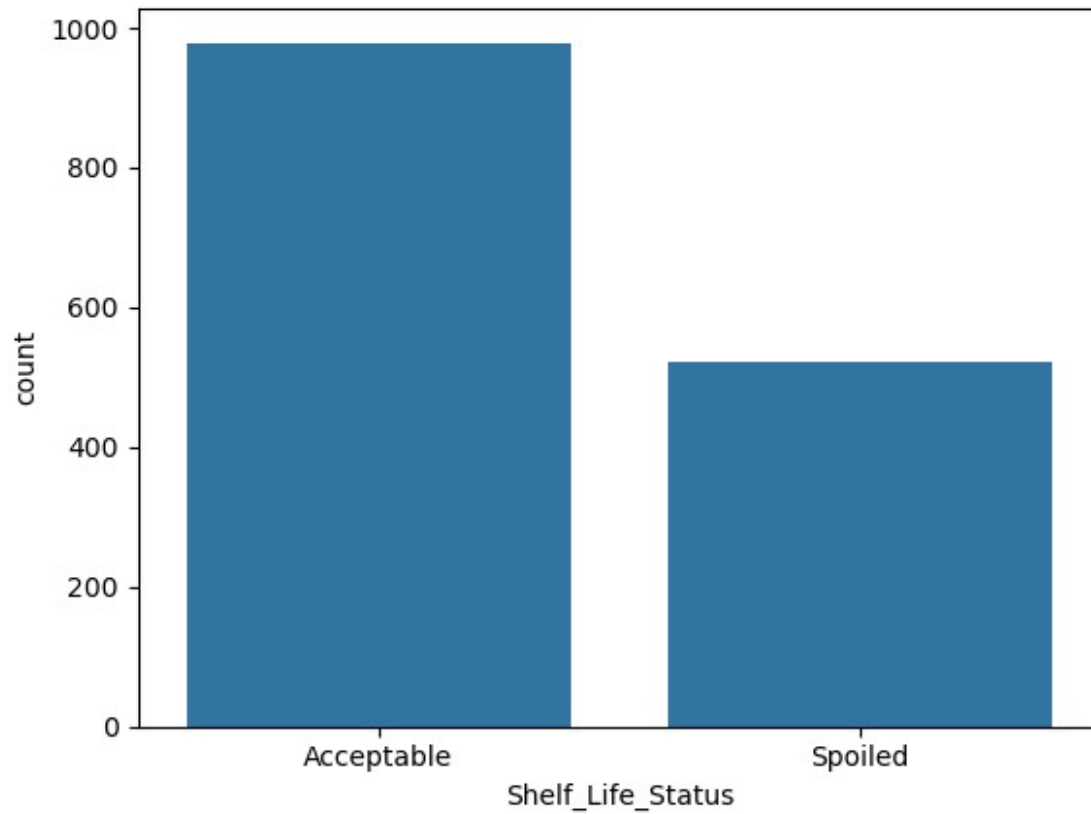
```
Storage_Temperature_C      0
Storage_Time_Days          0
pH                          0
Titratable_Acidity_pct     0
Microbial_Load_log_CFU_mL  0
Sensory_Score              0
Shelf_Life_Status          0
dtype: int64
Duplicates: 0
```

*# Step 5 – Check target distribution*

```
print(df["Shelf_Life_Status"].value_counts())
```

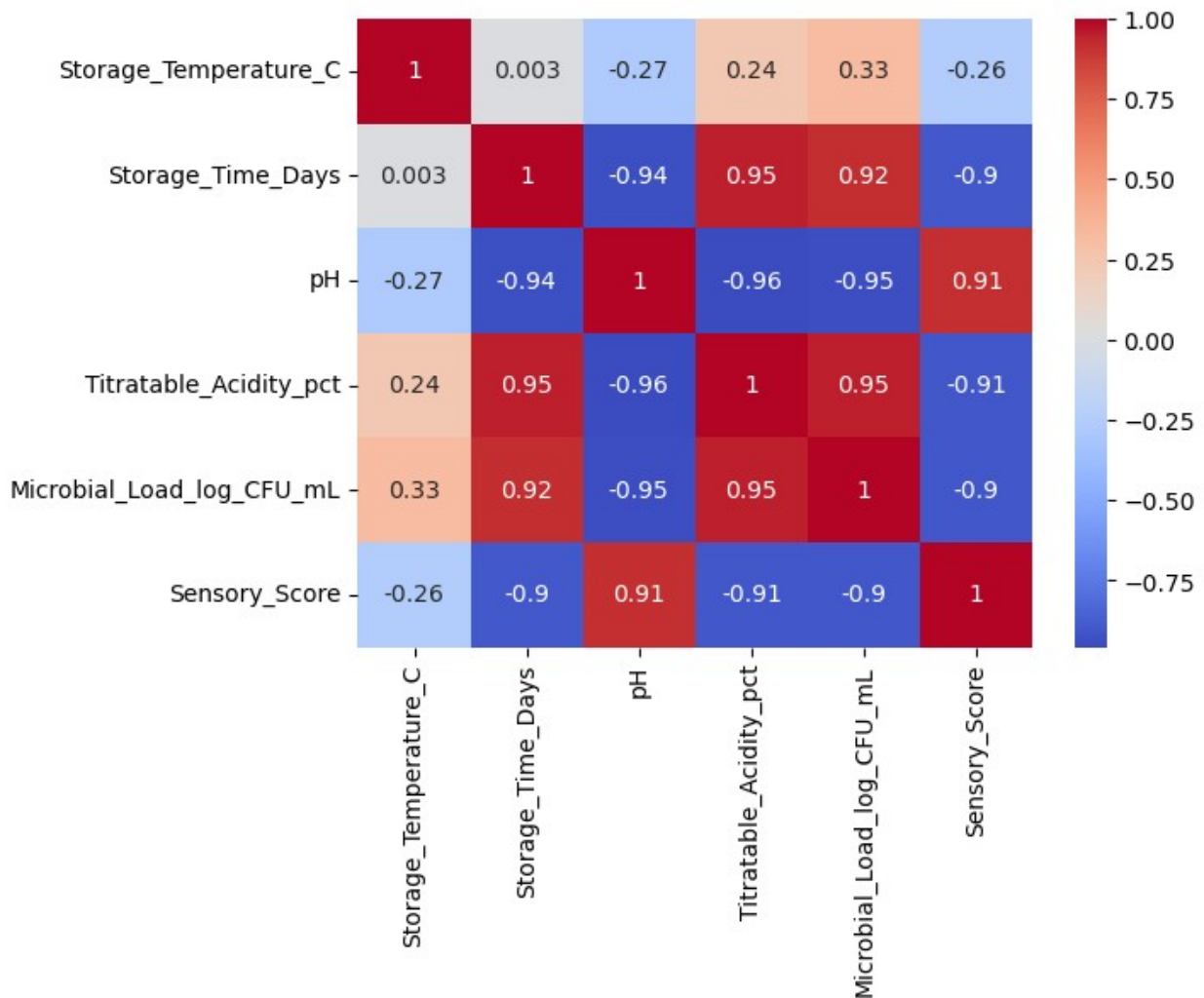
```
sns.countplot(x="Shelf_Life_Status", data=df)
plt.show()
```

```
Shelf_Life_Status
Acceptable      979
Spoiled         521
Name: count, dtype: int64
```



*# Step 6 – Correlation between numerical variables*

```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")  
plt.show()
```



```
# Step 7 – Select features and target
X = df.drop("Shelf_Life_Status", axis=1)
y = df["Shelf_Life_Status"]

# Step 8 – Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape)
print(X_test.shape)

(1200, 6)
(300, 6)

# Step 9 – Logistic Regression
```

```

model_lr = LogisticRegression(max_iter=1000)
model_lr.fit(X_train, y_train)
y_pred_lr = model_lr.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))

```

Accuracy: 0.9533333333333334

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Acceptable   | 0.95      | 0.98   | 0.96     | 192     |
| Spoiled      | 0.96      | 0.91   | 0.93     | 108     |
| accuracy     |           |        | 0.95     | 300     |
| macro avg    | 0.96      | 0.94   | 0.95     | 300     |
| weighted avg | 0.95      | 0.95   | 0.95     | 300     |

### # Step 10 – Decision Tree

```

model_dt = DecisionTreeClassifier(random_state=42)
model_dt.fit(X_train, y_train)
y_pred_dt = model_dt.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))

```

Accuracy: 0.9966666666666667

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Acceptable   | 0.99      | 1.00   | 1.00     | 192     |
| Spoiled      | 1.00      | 0.99   | 1.00     | 108     |
| accuracy     |           |        | 1.00     | 300     |
| macro avg    | 1.00      | 1.00   | 1.00     | 300     |
| weighted avg | 1.00      | 1.00   | 1.00     | 300     |

### # Step 11 – Random Forest

```

model_rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
model_rf.fit(X_train, y_train)
y_pred_rf = model_rf.predict(X_test)

```

```
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

Accuracy: 0.9966666666666667

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Acceptable   | 0.99      | 1.00   | 1.00     | 192     |
| Spoiled      | 1.00      | 0.99   | 1.00     | 108     |
| accuracy     |           |        | 1.00     | 300     |
| macro avg    | 1.00      | 1.00   | 1.00     | 300     |
| weighted avg | 1.00      | 1.00   | 1.00     | 300     |

*# Step 12 – Compare models*

```
results = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random
Forest"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_lr),
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_rf)
    ]
})

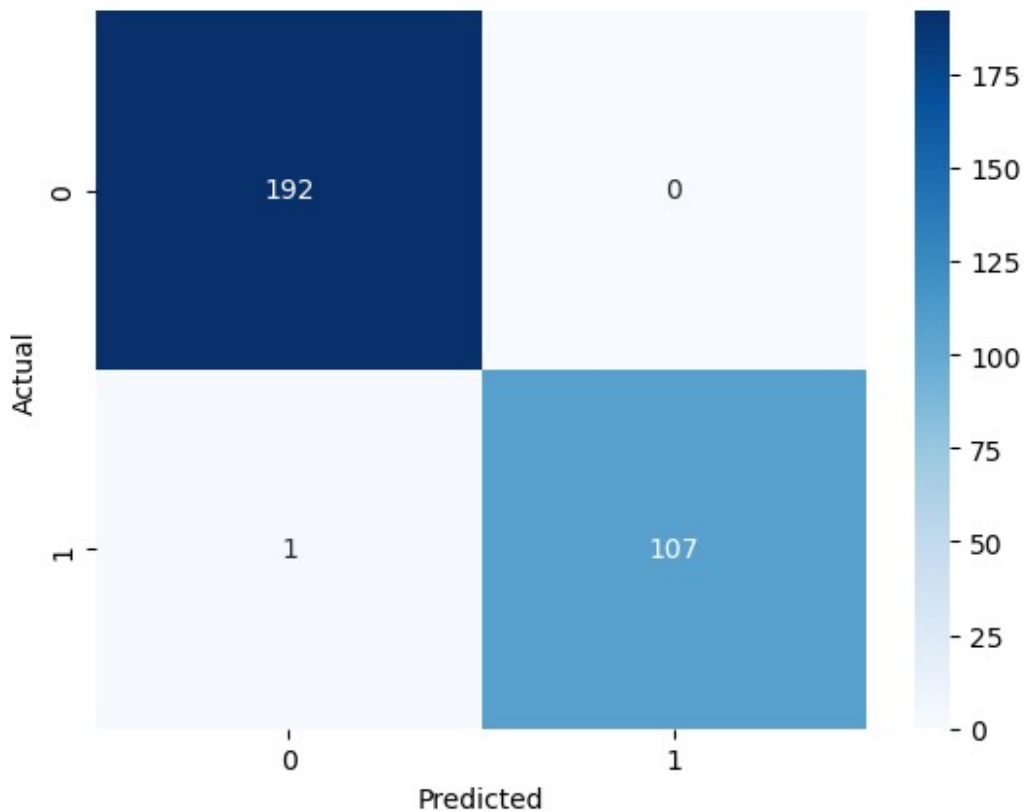
print(results)
```

|   | Model               | Accuracy |
|---|---------------------|----------|
| 0 | Logistic Regression | 0.953333 |
| 1 | Decision Tree       | 0.996667 |
| 2 | Random Forest       | 0.996667 |

*# Step 13 – Confusion Matrix*

```
cm = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



#### # Step 14 – Feature Importance

```
importance = pd.Series(
    model_rf.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print(importance)

importance.plot(kind="bar")
plt.ylabel("Importance")
plt.show()
```

|                           |          |
|---------------------------|----------|
| Titratable_Acidity_pct    | 0.441776 |
| pH                        | 0.182669 |
| Microbial_Load_log_CFU_mL | 0.154118 |
| Storage_Time_Days         | 0.145619 |
| Sensory_Score             | 0.068628 |
| Storage_Temperature_C     | 0.007190 |

dtype: float64

